# Memory Management Intro

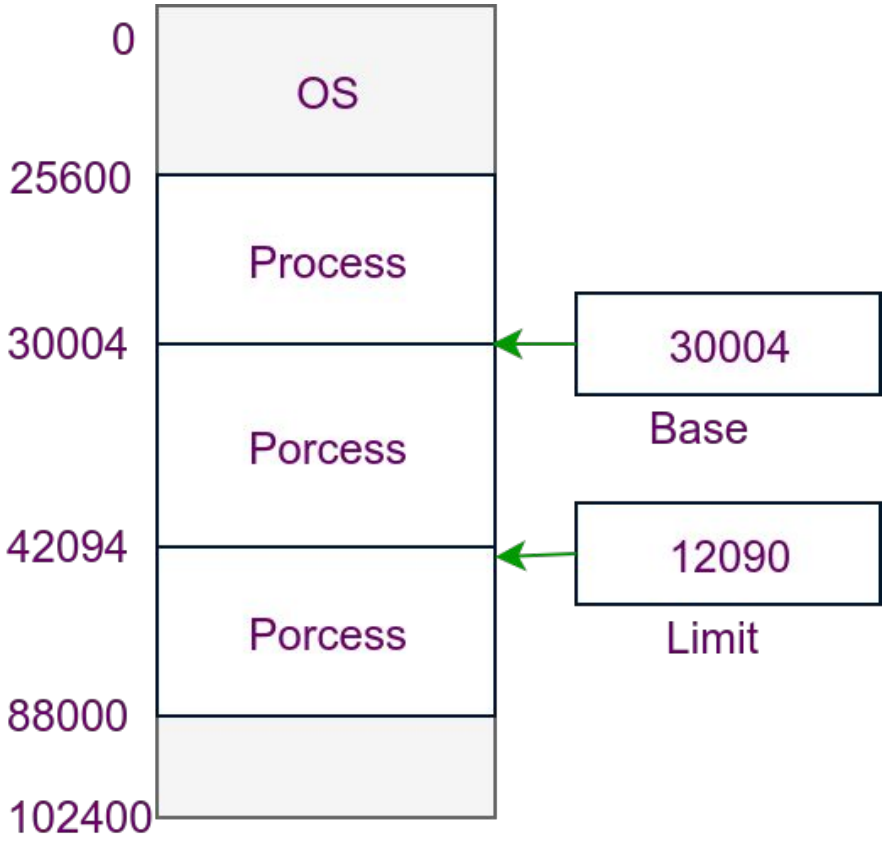
Dr. Jit Mukherjee

# Basic Hardware

- CPU can be shared by a set of processes
  - We must keep several processes in memory
- Memory consists of a large array of words or bytes with own address
- CPU fetches instructions from memory according to program counter
  - **Instruction-execution cycle** - fetch instruction from memory, decode instruction, may fetch some data from memory, results stored back in memory
  - The memory unit sees a stream of memory addresses
- Only storage CPU can access directly -> main memory and registers
  - Any instruction or data in execution must be in these
- Registers are fast -> CPU can access by one cycle
- Memory bus to access to main memory
  - Processor need to stall
- Remedy of this to have Cache memory.

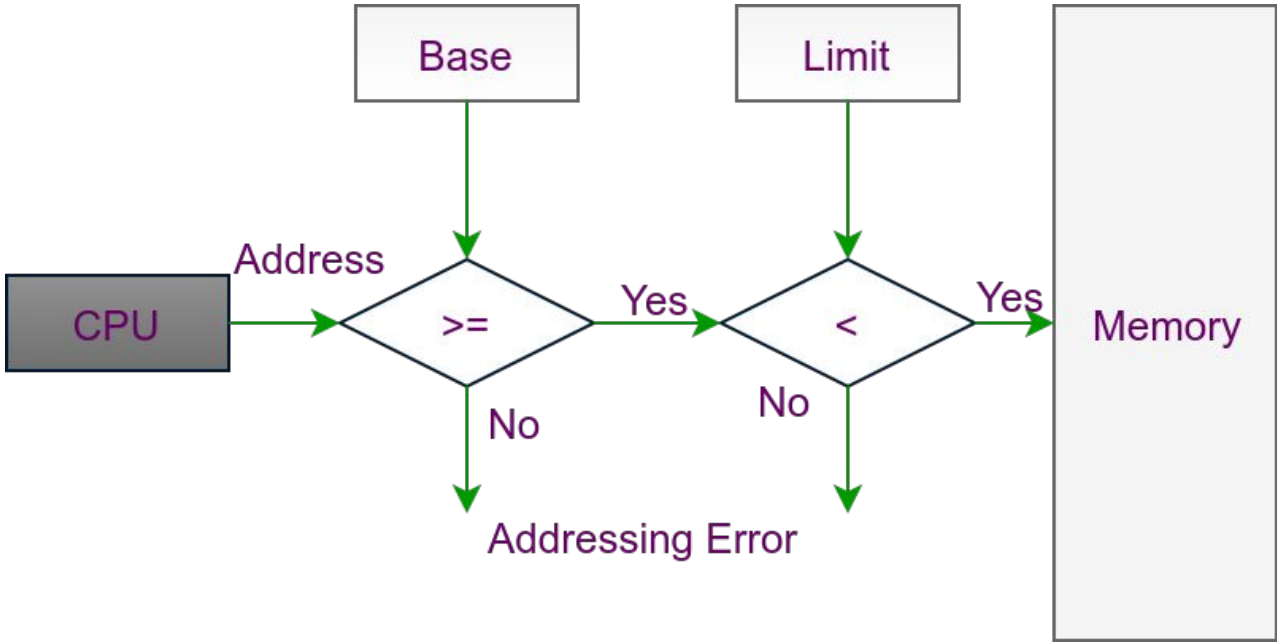
# Basic Hardware

- Protect OS memory from user processes access
- Protect user process from another
- Hardware implementation
  - Each process should have separate memory space
  - Range of legal address that process can access
- Base Register - Smallest legal physical memory address
- Limit Register - Size of the range
- Base - 300040, Limit - 120900



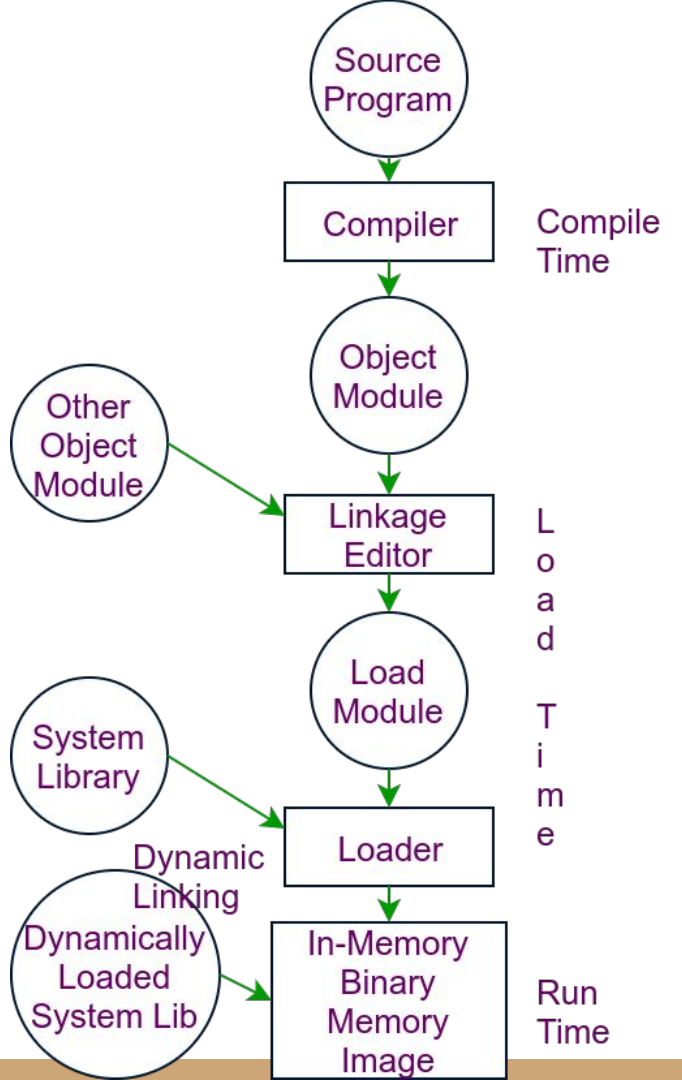
# Hardware Address Protection

- CPU hardware compare every address generated from user
- Any attempt to access OS memory or other user -> Trap
- Base and limit address loaded by OS - Special Privileged Instruction



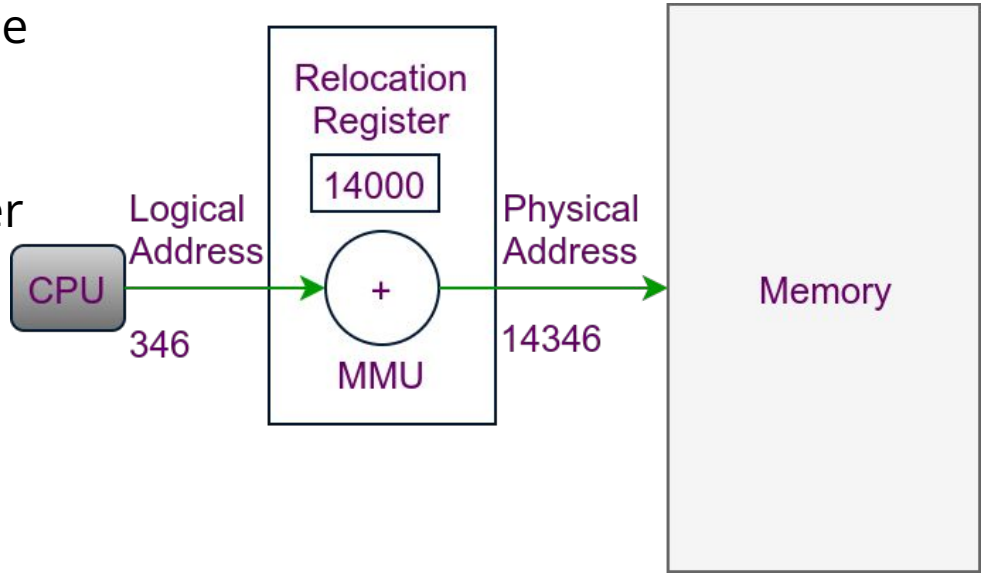
# Address Binding

- Program reside in disk - binary executable file
- To execute -> in memory
  - Input Queue -> Waiting in disk ti be in memory
- A process from queue -> execution -> access instruction and data from memory -> release
- Addresses in user programme - symbolic
  - Compiler bind them to relocatable address
  - 10 byte from the entry of a module
- Linkage editor and Loader - bind relocatable address to absolute address
- Binding - one address space to another
  - Compile Time - Absolute Code
  - Load Time - Relocatable Code
  - Execution TIme - Binding delayed until runtime



# Logical Vs Physical Address Space

- An address generated by CPU -> **Logical Address**. Address seen by memory unit -> loaded into memory-address register-> **Physical address**
- Compile and load time binding produces identical logical and Physical
- Execution time binding -> different virtual (logical) and physical address
- Logical and Physical address space
- Memory management unit
  - Run time mapping
- Base Register - Relocation Register
- User never sees real address
  - Deals with virtual address space
  - Had notion of running in virtual
- Logical Range -> [0, max]
  - Physical -> [R+0, R+max]



# Dynamic Loading

- Entire program and data should be in physical memory
  - Size of a process is limited to size of physical memory
- Better memory space utilization -> Dynamic Loading
  - A routine not loaded until it is called
  - All routn kept in disk in relocatable load format
- Main program is loaded and executed -> calls a routine
  - Checks if already loaded in main memory
  - Relocatable linking loader is called and updates address table
- Adv - unused routine will never be loaded. Large amount of code needed
- No special support from OS -> Developers responsibility

# Dynamic Linking and Shared Libraries

- Static linking - System language libraries -> any other object module -> combined into binary program image
- Dynamic Linking - linking is postponed until run time
  - Language subroutine libraries
  - Saves disk space and main memory
- A stub (*different from RMI*) - small piece of code
  - How to locate appropriate memory-resident library
  - How to load library if not already present
- Library update - library replaced by new version
  - Static Linking - relink
  - More than one version can be loaded in main memory
  - Shared libraries
- Dynamic linking needs help from OS
  - OS checks other process’s address space

# CPU Scheduling: A Recap

- Algorithm evaluation - what scheduling algo to choose
  - Maximizing CPU utilization under the constraint that the max response time is 1 Second
  - Maximizing Throughput - Turnaround time linearly proportional to total execution time
- Deterministic Modelling - Analytic Evaluation - Select all and check
- FCFS - average waiting time = 28 milliseconds
- SJF - average waiting time = 13 millisecond
- RR (10 quantum) - average waiting time = 23 milliseconds
- Queueing Model - nondeterministic -
  - studies the distribution of CPU and I/O burst
  - Knowing arrival rates and service rates
  - **Little's Formula** - > n = ƛ*W, n - average queue length,
  - ƛ - average arrival rate, W - average waiting time.
- Simulations - using simulators

| Process | Burst Time |
|---|---|
| P1 | 10 |
| P2 | 29 |
| P3 | 3 |
| P4 | 7 |
| P5 | 12 |